

TASK 3 REPORT

Machine Learning Model Deployment using FastAPI and Docker

Submitted By

Medarametla Yogendra
B.Tech – Artificial Intelligence & Machine Learning
Narasimha Reddy Engineering College

Internship Information

Program: Interspark AI/ML Internship Program
Domain: Artificial Intelligence & Machine Learning

Abstract

Model deployment is a critical stage in the machine learning lifecycle, enabling trained models to provide actionable predictions in real-world applications. This project demonstrates the deployment of an Iris Flower Classification model using FastAPI and Docker, adhering to industry-standard deployment practices.

A Random Forest Classifier was trained using the Iris dataset and exposed through a high-performance REST API capable of performing real-time inference. The implementation emphasizes scalability, portability, and professional software engineering standards.

Project Objectives

The primary objective of this project is to develop and deploy a machine learning model via a REST API utilizing FastAPI and modern containerization techniques to ensure environment consistency and ease of deployment.

Problem Statement

Machine learning models often remain confined to development environments, limiting their utility. To bridge the gap between development and production, models must be exposed through robust, scalable, and reusable interfaces that external applications can consume.

This project addresses the challenge of model accessibility by implementing a production-ready API service for a trained classifier.

Dataset Description

Dataset: Iris Flower Dataset

Source: Scikit-Learn

The dataset contains flower measurements from three iris species:

- Setosa
- Versicolor
- Virginica

Input Features:

- Sepal Length
- Sepal Width
- Petal Length
- Petal Width

Target Variable:

- Flower Species
-

Methodology

Data Preparation and Preprocessing

The Iris dataset was loaded using Scikit-Learn.

Data was divided into training and testing sets to evaluate model performance.

Model Training and Evaluation

Algorithm Used:

Random Forest Classifier

Reasons for Selection:

- High classification accuracy
 - Robust performance
 - Minimal preprocessing requirements
-

Model Serialization

The trained model was saved using Joblib:

```
joblib.dump(model, "Model/iris_model.pkl")
```

This enables loading the model for future inference without retraining.

API Architecture and Development

Framework:

FastAPI

 File

Figure 1: Swagger UI Documentation for API Endpoints

Endpoints Implemented:

GET /

Returns API status information.

POST /predict

Accepts flower measurements and returns predicted species.

Example Request:

```
{
  "sepal_length": 5.1,
  "sepal_width": 3.5,
  "petal_length": 1.4,
  "petal_width": 0.2
}
```

Example Response:

```
{
  "prediction": "Setosa"
}
```

📄 File

Figure 2: Real-time Prediction Request and Response

Containerization with Docker

A Dockerfile was created to package the application and its dependencies.

Containerization Benefits:

- Portability
- Consistent Environment
- Simplified Deployment
- Scalability

Docker Commands:

```
docker build -t iris-api .
```

```
docker run -p 8000:8000 iris-api
```

📄 File

Figure 3: Project Folder Structure and Dependencies

Results and Findings

The API successfully performed predictions through FastAPI endpoints.

Key Achievements:

- Model training completed successfully
- API endpoints operational
- Interactive Swagger documentation generated
- Docker deployment configuration created

Technical Discussion

The project demonstrates how machine learning models can be integrated into production systems through REST APIs. FastAPI simplified API development while Docker enabled deployment portability.

The architecture follows industry practices commonly used in modern AI and machine learning applications.

Conclusion

This project successfully deployed a machine learning model using FastAPI and Docker. The resulting API provides real-time predictions through a scalable and reusable interface.

The project enhanced understanding of model deployment, API development, and MLOps fundamentals.

Future Scope

- Cloud Deployment using AWS or Azure
 - CI/CD Pipeline Integration
 - Kubernetes Deployment
 - Authentication and Security
 - Monitoring and Logging
 - Database Integration
-

Project Repository

<https://github.com/Yogendra1823/Iris-Classification-API>

Formal Declaration

This project was developed independently as part of the Interspark AI/ML Internship Program for educational and internship submission purposes.

127.0.0.1:8000/docs#/default/predict_predict_post

Execute

Clear

Responses

Curl

```
curl -X 'POST' \
  'http://127.0.0.1:8000/predict' \
  -H 'accept: application/json' \
  -H 'Content-Type: application/json' \
  -d '{
    "sepal_length": 5.1,
    "sepal_width": 3.5,
    "petal_length": 1.4,
    "petal_width": 0.2
  }'
```

Request URL

http://127.0.0.1:8000/predict

Server response

Code	Details
200	Response body <pre>{ "prediction": "Setosa" }</pre> Download Response headers <pre>content-length: 23 content-type: application/json date: Tue, 02 Jun 2026 11:35:27 GMT server: unicorn</pre>

Responses

Iris Flower Classification API 1.0 OAS 3.1

/openapi.json

default

GET	/ Home
POST	/predict Predict

Schemas

HTTPValidationError > Expand all object

IrisInput > Expand all object

ValidationError > Expand all object